1. 迭代器

1.1 迭代器的概念

1.1.1 迭代器的定义

迭代器（Iterator）是一个实现了迭代器协议的**对象**。迭代器协议指的是对象需要实现两个方法：**__iter()__**和**__next()__**。

1. **__iter()__**方法：当迭代器被创建时，这个方法会被调用，并且应该返回迭代器对象本身。这个方法使得对象能够被用在for循环以及其他需要迭代器的上下文中。
2. **__next()__**方法：这个方法会在每次迭代中被调用，并且返回序列中的下一个元素。如果所有的元素都已经迭代完毕，会抛出一个**StopIteration**类型的异常，表示迭代已经完成。

1.1.2 可迭代对象的定义

可迭代对象（Iterable）是指能够返回一个**迭代器**的对象，换句话说，**可迭代对象**是实现了__iter()__方法的对象，具有以下特点：

1. **实现了__iter()__方法**：可迭代对象必须拥有该方法，以便于返回一个迭代器对象。
2. **可以用于for循环**：可迭代对象可以用于for循环，在for循环时，Python会自动调用__iter()__方法来获取迭代器，然后不断调用迭代器的__next()__去获取下一个元素。比如Python内置的列表、元组、字符串等都是可迭代对象，在使用for循环去遍历时，会调用可迭代对象的__iter()__方法来获取一个迭代器，接着不断调用迭代器的__next()__方法去获取下一个元素，直到元素全部迭代完毕，抛出异常，for循环会自己捕获这个异常并退出循环，从而完成一次for循环。

1.1.3 迭代器与可迭代对象的关系

1. **所有迭代器都是可迭代对象**：因为迭代器都必须具有__iter()__和__next()__方法，并且通常返回自身，也可以去访问序列中的下一个元素，并在所有元素都被迭代后抛出**StopIteration**异常。
2. **不是所有可迭代对象都是迭代器**：可迭代对象只实现__iter()__方法，并不实现__next()__方法。比如Python内置的列表、元组、字符串等虽然是可迭代对象，但不是迭代器。

华清远见云宇宙实验中心 www.yzlab.com.cn

华清远见 HQYJ.COM

```
总结：
1. 凡是可作用于for循环的对象都是可迭代对象。
2. 凡是同时拥有__iter()__和__next()__方法的都是迭代器。
```

1.2 内置类型的迭代器

对于Python内置的可迭代对象来说，可以直接通过调用每一个对象已经定义好的__iter()__方法来获取对应的迭代器，以列表举例：

```python
# 创建一个列表，这是一个内置的可迭代对象
my_list = [1, 2, 3, 4, 5]

# 使用 iter() 函数获取列表的迭代器
list_iterator = iter(my_list)

# 使用迭代器的 __next__() 方法来遍历列表
try:
    while True:
        # 获取下一个元素
        item = list_iterator.__next__()
        print(item)
except StopIteration:
    # 当没有更多元素时，迭代器会抛出 StopIteration 异常
    print("Iteration is complete.")
```

再比如字典：

```python
# 创建一个字典，这是一个内置的可迭代对象
my_dict = {1: 'one', 2: 'two', 3: 'three'}

# 使用 iter() 函数获取字典的迭代器
dict_iter = my_dict.__iter__()

# 使用迭代器的 __next__() 方法来遍历字典
try:
    while True:
        # 获取下一个元素
        item = dict_iter.__next__()
        print(item)
except StopIteration:
    # 当没有更多元素时，迭代器会抛出 StopIteration 异常
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
    print('Iteration is complete')
```

1.3 自定义迭代器

1.3.1 自定义迭代器的简单应用

在Python中也可以自定义自己的迭代器，只需要在构造类时实现**__iter()__**和**__next()__**两个方法即可，**__iter()__**和**__next()__**两个方法的定义如下所示：

```
class MyRangeIterator:  # 定义一个名为MyRangeIterator的类
    def __init__(self, end):  # 构造函数，初始化迭代器
        self.end = end  # 设置迭代器的结束值
        self.num = 0  # 设置当前迭代的数值，初始为0

    def __iter__(self):  # 实现__iter__方法，返回迭代器本身
        return self  # 因为我们的迭代器就是实例本身，所以返回self

    def __next__(self):  # 实现__next__方法，返回下一个值
        if self.num < self.end:  # 如果当前数值小于结束值
            value = self.num  # 获取当前数值作为要返回的值
            self.num += 1  # 将当前数值加1，为下一次迭代做准备
            return value  # 返回当前数值
        else:  # 如果当前数值已经达到或超过结束值
            raise StopIteration  # 抛出StopIteration异常，结束迭代

# 使用MyRangeIterator类创建一个迭代器实例，迭代0到4的整数
for i in MyRangeIterator(5):  # for循环会自动调用迭代器的__next__方法
    print(i)  # 打印迭代器返回的每个值
```

1.3.2 斐波那契数列

定义如下：

1. 数列的前两个数是0和1。
2. 从第三个数开始，每个数都是前两个数的和。比如0，1，1，2，3，5，8，...

使用迭代器去生成指定个数的斐波那契数列，如下所示：

```
class FibonacciIterator:  # 定义一个名为FibonacciIterator的类
    def __init__(self, n):  # 构造函数，初始化迭代器
        self.n = n  # 设置迭代器的结束值
```

华清远见开发实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
        self.a, self.b = 0, 1  # 初始化斐波那契数列的前两个数
        self.count = 0  # 初始化计数器，用于跟踪迭代次数

    def __iter__(self):  # 实现__iter__方法，返回迭代器本身
        return self  # 因为我们的迭代器就是实例本身，所以返回self

    def __next__(self):  # 实现__next__方法，返回下一个值
        if self.count < self.n:  # 如果当前计数器小于结束值
            value = self.a  # 获取当前斐波那契数作为要返回的值
            self.a, self.b = self.b, self.a + self.b  # 更新斐波那契数列的下一个数
            self.count += 1  # 增加计数器
            return value  # 返回当前斐波那契数
        else:  # 如果当前计数器已经达到或超过结束值
            raise StopIteration  # 抛出StopIteration异常，结束迭代

# 使用FibonacciIterator类创建一个迭代器实例，迭代生成前10个斐波那契数
fib_iterator = FibonacciIterator(10)  # 创建一个生成前10个斐波那契数的迭代器

for number in fib_iterator:  # for循环会自动调用迭代器的__next__方法
    print(number)  # 打印迭代器返回的每个值
```

2. 生成器

2.1 生成器的概念

生成器（Generator）是一种特殊的迭代器，它可以在需要时动态生成值，避免一次性生成所有值所占用的大量内存。生成器使用 yield 语句来产生值，可以通过迭代器协议来逐个获取生成器产生的值。生成器可以在循环中被逐个迭代，从而实现高效地处理大量数据或者无限序列。在Python中，生成器可以通过函数定义和生成器表达式来创建。

2.2 生成器的特点

生成器的主要特点包括：

1. **惰性求值**：生成器不会在创建时生成所有值，而是逐个生成值。这意味着生成器可以在需要的时候再去生成值。

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

2. **使用yield关键字**：只有**yield**被调用时，生成器才会返回一个值，并且程序暂停执行，直到下一次迭代时才继续执行。
3. **内存效率**：生成器可以处理大型数据集或生成大型数据结构，而不会占用太多内存，因为它们只在需要时生成值。
4. **迭代器协议**：生成器也是迭代器，所有实现了迭代器协议，因此可以用于for循环和其他需要迭代器的代码中。

2.3 生成器函数

生成器使用最多的地方是和函数结合，因此，拥有生成器的函数就叫做生成器函数，也可以理解为函数中包含了**yield**关键字的函数就是一个生成器函数。生成器函数包含如下特点：

1. **使用yield关键字**：普通函数使用return返回值，而生成器函数使用**yield**关键字返回值。
2. **状态保持**：生成器函数在每次产生一个值之后会暂停执行，并保持当前的状态，包括局部变量的状态和当前的执行位置。当再次调用生成器时，他会从上次**yield**语句之后的地方继续执行。
3. **惰性求值**：生成器只在需要时才计算并生成值，这意味着它可以高效处理大量数据或无限流数据。

生成器函数在调用时，不会立即执行函数里面的代码，而是会先返回一个生成器的对象，当调用生成器对象的**__next()__**方法时，才会进入函数中并执行函数里面的代码，遇到**yield**关键字后，函数的执行将会暂停，并将**yield**后面的表达式作为当前迭代的值返回。然后每次调用生成器的**__next()__**方法或使用for循环进行迭代时，函数会从上次暂停的地方继续执行，直到再次遇到**yield**关键字，并再次返回一个值，直到无法继续生成为止。

2.4 自定义生成器函数

2.4.1 自定义生成器函数的简单应用

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```python
# 定义生成器函数
def count_up_to(max):
    count = 1
    while count <= max:
        yield count
        count += 1

# 使用生成器函数
for number in count_up_to(5):
    print(number)
```

2.4.2 无限序列生成

```python
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

# 打印前10个斐波那契数
fib = fibonacci()
for _ in range(10):
    print(next(fib))
```

华清远见元宇宙实验中心 yyzlab.com.cn